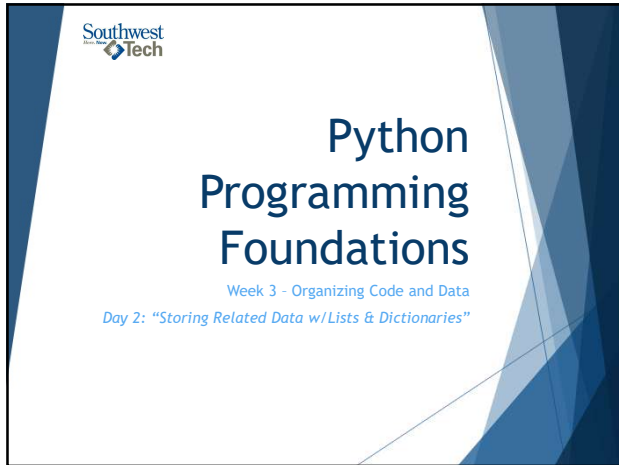
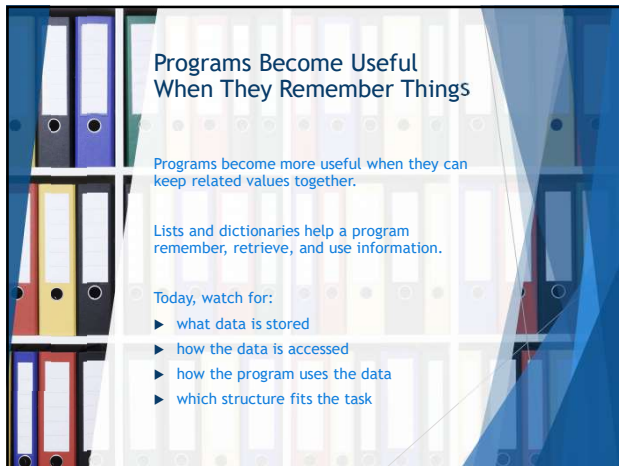
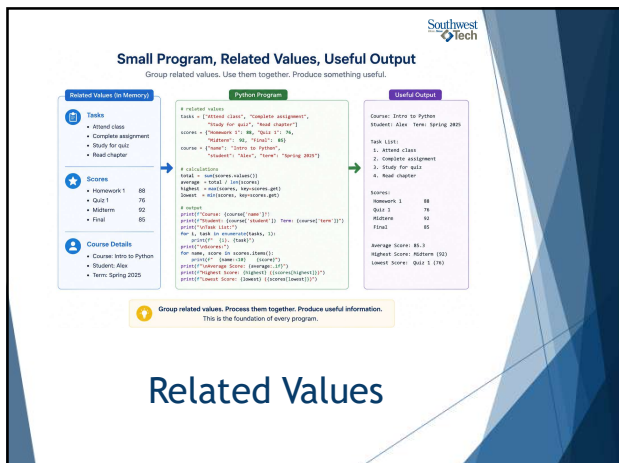




1



2



3



Logic Uses Data

Functions organize logic.
Lists and dictionaries organize information.

- ▶ Small programs often need both:
 - ▶ a function for the job
 - ▶ a data structure for the values
 - ▶ output that shows the result

4



Function Uses Data to Produce a Result

logic + related data → visible result

logic

`f()` `calculate_average(scores)`
Find the average of the scores.

Steps (Hidden Inside the Function)

1. Add up all the scores.
2. Count how many scores there are.
3. Divide total by count.
4. Return the average.

related data

scores (list)
[88, 76, 92, 85]

OR

student (dictionary)
{
 "name": "Alex",
 "scores": [88, 76, 92, 85],
 "term": "Spring 2023"
}

visible result

Average Score
85.25

Based on the scores:
[88, 76, 92, 85]

★ Logic does the work. Related data provides the values. A visible result is produced.
Change the data, get a new result.

Logic Plus Data

5

What Counts as Success Today

Store related data ->
retrieve useful values ->
loop or look up ->
show output.



For each data structure, ask:

What values are stored?

How do I access one value?

How do I use all values?

What visible behavior proves it works?

6

Southwest
Tech

From Data to Output: A Simple 4-Step Flow

Small program. Clear steps. Useful result.

1 Store related data

Keep related values together in a list or dictionary.

scores [list]
[88, 76, 92, 85]

→

2 Retrieve useful values

Get the values you need from the data.

total = 341
count = 4
average = 85.25

→

3 Loop or look up

Loop through items or look up information as needed.

Processing scores...
88 = added
76 = added
92 = added
85 = added

→

4 Show output

Present the result in a clear, readable way.

Results
Total: 341
Count: 4
Average: 85.25

✓ Good programs follow clear steps from data to output.
Change the data, run again, get a new result.

Today's Success

7

Southwest
Tech

What We Will Use Today

- ▶ lists
- ▶ indexes
- ▶ 'append()'
- ▶ loops over lists
- ▶ dictionaries
- ▶ keys and values
- ▶ dictionary lookup

These tools help a program work with more than one related value.

```
mirror_mod = modifier_ob
mirror_object = mirror
mirror_mod.mirror_object =
operation == "MIRROR_X"
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
operation == "MIRROR_Y"
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
operation == "MIRROR_Z"
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True
selection at the end - add
obj.select(1)
obj.select(1)
context.scene.objects.active
("Selected" + str(modifier
mirror_object.select(0)
bpy.context.selected_obj
data.objects[one.name].sel
int("please select exactly
--- OPERATOR CLASSES ---
types.Operator):
X mirror to the selected
object.mirror_mirror_X"
mirror X"
context:
next.active_object is no
```

8

Southwest
Tech

Python Data Toolbox

Simple concepts. Powerful tools.

list

Ordered collection of items.

fruits = ["apple", "banana", "carrot"]

→

index

Position of an item. It's a list starts at 0.

fruits[0] # "apple"
fruits[2] # "carrot"

→

append

Add an item to the end of a list.

fruits.append("pear")
["apple", "banana", "carrot", "pear"]

→

loop

Repeat actions for each item in a list.

for fruit in fruits:
print(fruit)

→

dictionary

Collection of key-value pairs.

person = {"name": "Hoyt", "age": 28}

→

key

Unique label used to find a value.

"name" # key in person

→

value

The data associated with a key.

person["name"] # "Hoyt"

→

lookup

Use a key to find the related value.

person["age"] # 28

★ These tools work together to store, find, and use data.
Practice with small examples. Build confidence step by step.

Toolbox

9

Southwest Tech

What We Will “Skip” For Now

- ▶ Comprehensions
- ▶ Generators
- ▶ Advanced collections
- ▶ Databases
- ▶ Complex nested data



10

Southwest Tech

Parked for Later

Useful later, not today's focus.

comprehensions	generators	advanced collections	databases	complex nested data
Compact ways to create lists, sets, or dictionaries. <code>squares = [x**2 for x in range(10)]</code>	Produce values one at a time on demand. <code>def count(): n = 0 while True: yield n</code>	Specialized containers for specific tasks. <code>from collections import deque, defaultdict</code>	Store and query large amounts of persistent data. <code>SELECT * FROM students WHERE grade > 90;</code>	Densely nested structures and relationships. <code>{ "user": { "name": "Alice", "roles": [...] }}</code>

▶ We'll come back to these when you're ready. ▶

★ Master the basics first. These topics build on that foundation. You'll use them at the right time.

Parked For Later

11

Southwest Tech

Lists Keep Related Items Together

A list stores multiple related values in one variable.

Example:

```
1 tasks = ["read", "practice", "submit"]
2
3
```

Use a list when order or repeated display matters.

12

Southwest
Tech

A List of Tasks in Python

A list groups related items together in order.

one list variable

```
tasks = [
```

 **read**
Read the chapter for class.

 **practice**
Practice the exercises.

 **submit**
Submit the assignment before the deadline.

```
]
```

★ The variable `tasks` holds a list with three items: `read`, `practice`, `submit`.

List Container

13

Southwest
Tech

List Positions Start At Zero

List items are accessed by position.

Python starts counting positions at '0'.

Example:

```

1  tasks = ["read", "practice", "submit"]
2
3  value = tasks[0] ←
4
5

```

This gets the first task.

14

Southwest
Tech

Python List Indexing (Zero-Based)

Indices start at 0.

```
tasks = ["read", "practice", "submit"]
```

index 0

 **read**

index 0

index 1

 **practice**

index 1

index 2

 **submit**

index 2

first item →

★ Zero-based indexing means the first item is at index 0.
Index 0 → first, Index 1 → second, Index 2 → third.

Zero-Based Index

15



16

A presentation slide titled "What to Watch For" with the Southwest Tech logo in the top right. The background shows a magnifying glass over a bar chart with labels "Q2" and "Q3".

Watch the list change.

Trace:

- ▶ the starting values
- ▶ the item accessed by position
- ▶ the loop that displays all items
- ▶ the new item added with `append()`

17

A presentation slide titled "Lists Grow with append()" with the Southwest Tech logo in the top right. It illustrates how a list grows by adding an item to the end.

BEFORE
tasks (3 items)
1. read
2. practice
3. submit

APPEND ACTION
`tasks.append("review")`
A new item is added to the end.

AFTER
tasks (4 items)
1. read
2. practice
3. submit
4. review

LOOP OUTPUT
Looping through the list shows each task in order:
1. read
2. practice
3. submit
4. review

Lists keep items in order. `append()` adds to the end.

List Demo

18

Southwest
Tech

Looping Makes A List Useful

A list becomes useful when the program can use each item.

Common beginner pattern:

```
1 tasks = ["read", "practice", "submit"]
2
3
4 for task in tasks:
5     print(task)
6
```

The loop handles one item at a time.

19

Southwest
Tech

Dictionaries



20

Southwest
Tech

Dictionaries Connect Meaning To Value

A dictionary stores values by key.

Example:

```
1 course = {"title": "python Programming", "credits": 2}
2
3
```

The key is a meaningful label for the value.

21

Southwest
Tech

Python Dictionary: Key / Value Pairs

A dictionary stores related data using meaningful keys.

```
course = { "number": "CS 101", "title": "Intro to Python", "credits": 3 }
```

Key (meaningful name)	Value (the data)
number	"CS 101"
title	"Intro to Python"
credits	3

Keys are unique. Values can be any type (text, number, and more).

Dictionary Pairs

22

Southwest
Tech

Keys Are Not Positions

- ▶ Lists use **positions**.
- ▶ Dictionaries use **keys**.

Compare:

```
1 tasks = ["read", "practice", "submit"]
2 course = {"title": "Python Programming", "credits": 2}
3
4 tasks[0] # Accessing the first task
5 course["title"] # Accessing the course title
6
7
```

The list asks for a position. The dictionary asks for a label.

23

Southwest
Tech

Accessing Data: Position vs Key

position

List access uses a numeric position.

```
tasks[0]
```

tasks (list)

0	read
1	practice
2	submit

Result: read

We use a number (position) to get an item from a list.

key

Dictionary access uses a meaningful key.

```
course["title"]
```

course (dictionary)

number	CS 101
title	Intro to Python
credits	3

Result: Intro to Python

We use a meaningful key (name) to get a value from a dictionary.

Position vs. Key

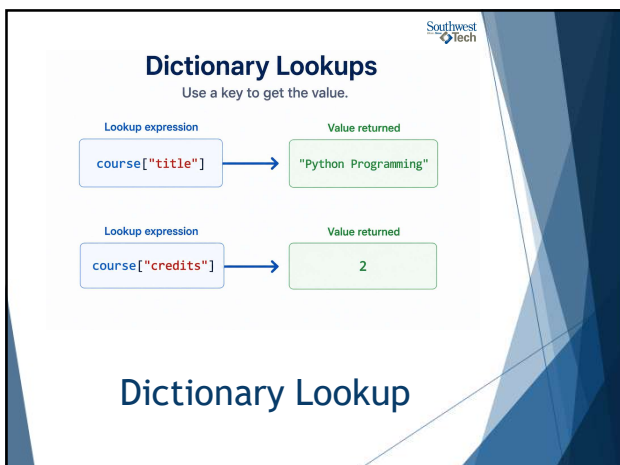
24



25



26



27



Choosing A Structure

Use a list when:

- ▶ You have several related items
- ▶ Order matters
- ▶ You want to loop through all items

Use a dictionary when:

- ▶ Labels matter
- ▶ You want to look up a value by name
- ▶ One item has several named parts

28



Choose Structure

Use a list when order or looping matters

Lists keep items in a specific order. Great for steps, sequences, and loops.

Example: tasks (list)

0		read
1		practice
2		submit
3		review

Loop example:
Visit each task in order:
read → practice → submit → review

Use a dictionary when labels matter

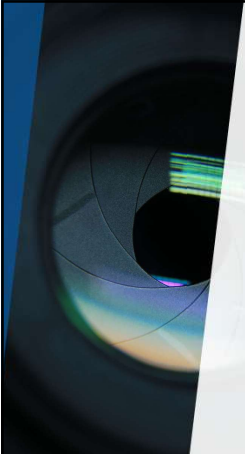
Dictionaries store labeled data. Great for lookups by meaningful names.

Example: course (dictionary)

title	→	Python Programming
number	→	CS 101
credits	→	2
instructor	→	A. Smith

Lookup example:
Get values by name:
course["title"] → "Python Programming"
course["credits"] → 2

29



Demo #3

"List of Dictionaries"

30



What to Watch For

A list of dictionaries can store multiple records.

In the demo, each student has:

- ▶ a name
- ▶ a score
- ▶ a status decision

The loop handles one record at a time.

31



Lists Can Hold Structured Records

A list can hold many small, related records.

students (list of records)

Record 0	Record 1	Record 2
name: Maya	name: Jordan	name: Luis
score: 92	score: 85	score: 78

Loop idea: Visit one record at a time.

```

graph LR
    R0[Record 0: name: Maya, score: 92] --> R1[Record 1: name: Jordan, score: 85]
    R1 --> R2[Record 2: name: Luis, score: 78]
  
```

This is a bridge to future structured data. You don't need this right now, but lists of records are a powerful next step.

Demo 3: List of Dictionaries

32



Assignment #5

Build one small mini-app using a list or dictionary.

A good first version should include:

- ▶ one chosen structure
- ▶ a few sample values
- ▶ one useful lookup or loop
- ▶ visible output
- ▶ a short explanation of why the structure fits

33

Southwest
Tech

Mini-App Ideas: Pick Any One

Different apps, different data structures, visible results.

Task List
Manage your tasks

Data structure: list

```
["read", "practice", "submit"]
```

↓

Visible output

My Tasks

- ☐ read
- ☐ practice
- ☐ submit

Simple Inventory
Track items and counts

Data structure: dictionary

```
{ "apple": 4, "book": 2, "bottle": 1 }
```

↓

Visible output

Item	Count
apple	4
book	2
bottle	1

Score Tracker
Record and view scores

Data structure: list of records

```
[{"name": "Maya", "score": 92}, {"name": "Jordan", "score": 85}, {"name": "Luks", "score": 78}]
```

↓

Visible output

Name	Score
Maya	92 ★
Jordan	85 ★
Luks	78 ★

Course Lookup
Look up course info

Data structure: dictionary

```
{ "title": "Python 101", "credits": 2, "instructor": "A. Smith" }
```

↓

Visible output

Title	Python 101
Credits	2
Instructor	A. Smith

Each mini-app helps you practice core ideas. Choose the one that interests you most.

All are great ways to learn. None is required.

Assignment #5

34

Southwest
Tech

Evidence for A Data Mini-App



Your submission should show that the data structure is useful.

Useful evidence includes:

- ▶ one working `.py` file
- ▶ list or dictionary data
- ▶ retrieval, lookup, or loop behavior
- ▶ visible output
- ▶ short explanation of the structure choice

This Photo by Unknown Author is licensed under CC BY-SA

35

Southwest
Tech

Evidence: One Program, Clear Data, Visible Result

Small, clear pieces that show your understanding.

1. Program File tasks.py

```
1 tasks = ["read", "practice", "submit", "review"]
2
3 print("My Tasks:")
4
5 for task in tasks:
6     print(f"- {task}")
7
8
9
```

2. Data (List)

tasks (list)

index 0: read

index 1: practice

index 2: submit

index 3: review

3. Output (Sample)

My Tasks:

- read
- practice
- submit
- review

4. Structure Choice

A list fits because order matters and we need to loop through each task.

✔ This matches the problem and produces the results above.

Evidence

36



37
